



Portfolio

Internationale Hochschule Fernstudium
Studiengang Master of Science – Informatik

MedicalTracker: Konzeption, Softwarearchitektur und Implementierung eines digitalen Assistenzsystems für den Pflegealltag

Modul:	Projekt: Software Engineering (DLMCSPSE01_D)
Tutor:	Prof. Dr. Dirk Simon
vorgelegt von:	Steffen Niklas Oehler
Datum:	26.07.2026
Matrikelnummer:	IU14142306
E-Mail:	steffen-niklas.oehler@iu-study.org

Inhaltsverzeichnis

Inhaltsverzeichnis	II
Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Glossar	V
1 Projektdokumentation	1
1.1 Projektübersicht.....	1
1.2 Technologie & Systemarchitektur	1
1.3 Risikomanagement.....	2
1.4 Zeitplanung.....	3
2 Anforderungsdokumentation.....	5
2.1 Stakeholder-Analyse	5
2.2 Funktionale Anforderungen	6
2.3 UML-Anwendungsfalldiagramm.....	8
2.4 Nichtfunktionale Anforderungen.....	8
3 Spezifikationsdokumentation	10
3.1 Objektorientiertes Datenmodell & Geschäftsobjekte	10
3.2 Geschäftsprozesse.....	12
3.3 Geschäftsregeln	14
3.4 Schnittstellen & Validierung	16
3.4.1 Systemschnittstellen	16
3.4.2 Benutzerschnittstellen & UI-Konzept	17
3.4.3 Mehrstufiges Validierungskonzept.....	20
Literaturverzeichnis.....	21

Abbildungsverzeichnis

Abbildung 1 Anwendungsfalldiagramm	8
Abbildung 2 Klassendiagramm.....	11
Abbildung 3 Aktivitätsdiagramm	13
Abbildung 4 Verwaltungsdashboard.....	18
Abbildung 5 Patientensicht.....	19

Tabellenverzeichnis

Tabelle 1 Risikoanalyse	2
Tabelle 2 Zeitplanung	3
Tabelle 3 Stakeholder-Analyse	5
Tabelle 4 Nichtfunktionale Anforderungen	9

Glossar

Begriff	Definition
Compliance (Medizin)	Die Zuverlässigkeit und Einhaltung der verordneten medizinischen Behandlung (z. B. pünktliche Einnahme von Medikamenten) durch die zu pflegende Person.
Flask	Ein leichtgewichtiges WSGI-Webanwendungs-Framework in Python, das nach dem Micro-Framework-Ansatz aufgebaut ist.
Jinja2	Eine weit verbreitete Template-Engine für Python, die zur dynamischen Generierung von HTML-Seiten im Flask-Backend genutzt wird.
MVP (Minimum Viable Product)	Das minimal funktionsfähige Produkt, das die Kernfunktionen (Medikation & Vitaldaten) erfüllt, um frühzeitig evaluierbar zu sein.
ORM (Object-Relational-Mapping)	Eine Technik der Softwareentwicklung, die Daten zwischen einer relationalen Datenbank (SQLite) und objektorientierten Programmiersprachen (Python) konvertiert (hier umgesetzt via SQLAlchemy).
Vitaldaten	Tägliche, messbare Körperfunktionen (z. B. Blutdruck, Puls, Körpergewicht, Blutzucker), die Aufschluss über den Gesundheitszustand liefern.

1 Projektdokumentation

Das erste Kapitel etabliert den organisatorischen und methodischen Rahmen für die Entwicklung der Anwendung MedicalTracker. Nach einer einleitenden Problemabgrenzung und Darstellung des angestrebten Kundennutzens wird die gewählte Systemarchitektur auf Basis von Python, Flask und SQLite begründet. Anschließend werden die Projektrisiken systematisch analysiert sowie der zeitliche Ablauf und die wesentlichen Arbeitspakete des Entwicklungsvorhabens strukturiert dargestellt.

1.1 Projektübersicht

Die häusliche Pflege von Angehörigen stellt für viele Menschen eine erhebliche organisatorische und emotionale Herausforderung dar. Die Koordination von Medikationsplänen, das regelmäßige Erfassen von Vitaldaten sowie die Vorbereitung auf Arzttermine erfolgen in der Praxis häufig noch auf Papier oder in unstrukturierten Notizen. Dies erhöht das Risiko von Fehldosierungen, vergessenen Einnahmen und Informationsverlusten bei medizinischen Konsultationen.

Das Projekt adressiert diese Problematik durch die Konzeption und Implementierung einer intuitiven, lokalen Webanwendung. MedicalTracker dient als zentrales digitales Assistenzsystem für den Pflegealltag. Die Anwendung verfolgt das Ziel, pflegende Angehörige bei der täglichen Dokumentation und Planung zu entlasten und zu pflegenden Personen durch eine barrierefreie Tagesansicht mehr Selbstständigkeit zu ermöglichen.

1.2 Technologie & Systemarchitektur

Zur Gewährleistung einer hohen Erfüllung der funktionalen und nichtfunktionalen Anforderungen sowie zur Vermeidung technischer Risiken wird für die Anwendung eine klassische Client-Server-Architektur auf Basis eines reinen Python-Ökosystems gewählt:

Backend

Programmiersprache Python (v3.13+) in Kombination mit dem leichtgewichtigen Web-Framework Flask. Flask ermöglicht eine saubere objektorientierte Strukturierung der Anwendungslogik und bietet maximale Kontrolle über Routing und Datenverarbeitung.

Frontend

Die Benutzeroberfläche wird mittels der Template-Engine Jinja2 serverseitig gerendert. Das UI-Styling basiert auf Bootstrap 5, wodurch ein modernes, responsives und barrierefreies Layout ohne den Overhead externer JavaScript-Frameworks realisiert werden kann.

Datenhaltung

Als Datenbanksystem kommt SQLite zum Einsatz. SQLite arbeitet dateibasiert, erfordert keine Serverinstallation beim Endanwender und ermöglicht eine datenschutzkonforme, lokale Datenspeicherung. Die Anbindung an das Python-Backend erfolgt isoliert über den Objekt-Relationalen Mapper (ORM) Flask-SQLAlchemy.

1.3 Risikomanagement

Zur frühzeitigen Identifikation und Reduktion von Projektrisiken wird eine systematische Risikoanalyse durchgeführt. Die Risiken werden nach Eintrittswahrscheinlichkeit und Auswirkung bewertet und mit konkreten Gegenmaßnahmen (Mitigation) versehen.

Tabelle 1 Risikoanalyse

Risiko-ID	Beschreibung	Eintrittswahrscheinlichkeit	Auswirkung	Gegenmaßnahme
R-01 (Technisch)	Datenverlust / -korruption Plötzlicher Serverabsturz beschädigt die dateibasierte SQLite-Datenbank.	Gering	Hoch	Einsatz von SQLAlchemy für sicheres Transaktionsmanagement; automatisierte Erstellung eines Backups der .db-Datei bei jedem App-Start.
R-02 (Design)	Mangelnde Usability (Barrierefreiheit) Die ältere Zielgruppe (Patienten) scheitert an zu kleinen Klickelementen oder komplexer Menüführung.	Mittel	Hoch	Konsequenter „Mobile-First“- und High-Contrast-Ansatz mit Bootstrap 5; Erstellung von UI-Prototypen vor der finalen Programmierung.
R-03 (Ressourcen)	Verzögerungen durch externe Faktoren Berufliche Spitzenbelastungen blockieren die Weiterentwicklung innerhalb des Zeitplans.	Mittel	Mittel	Definition eines Minimal Viable Products (MVP). Kernfunktionen (Medikation & Vitalwerte) besitzen höchste Priorität; optionale Features (PDF-Export) dienen als Puffer.

Quelle: eigene Darstellung

Die Auswertung der Risikomatrix zeigt, dass insbesondere das Design-Risiko (R-02) eine hohe Auswirkung auf den Gesamterfolg und den Kundennutzen der Anwendung hat. Durch die frühzeitige Berücksichtigung von Richtlinien zur Barrierefreiheit wird diesem Risiko bereits im Entwurf gezielt entgegengewirkt.

1.4 Zeitplanung

Die zeitliche Strukturierung des Projekts orientiert sich an einer veranschlagten Netto-Projektlaufzeit von 8 Wochen. Der Zeitplan verteilt die wesentlichen Arbeitspakete auf die drei vorgegebenen Phasen des Portfoliokurses.

Tabelle 2 stellt die Zeitplanung, die zugehörigen Arbeitspakete sowie die zentralen Qualitäts-Meilensteine (Einreichungen) dar.

Tabelle 2 Zeitplanung

Phase	Zeitraum	Wesentliche Arbeitspakete & Meilensteine
Phase 1: Konzeption	Woche 1-2	• Erstellung der Projektdokumentation (Einleitung, Risikomanagement, Zeitplan) • Erstellung des Anforderungsdokuments (Stakeholder, User Stories, NFAs, Glossar) • Erstellung des Spezifikationsdokuments (Datenmodell, Geschäftsprozesse, Schnittstellen) • Modeling der UML-Diagramme (Use-Case, Klasse, Aktivität) • Meilenstein 1: Abgabe Portfolioteil 1 & Feedback-Schleife
Phase 2: Architektur & Implementierung	Woche 3-6	• Erstellung des Architekturdokuments (Technologieentscheidung, UML-Sequenzdiagramm) • Aufsetzen des GitHub-Repositories und der Ordnerstruktur • Backend-Implementierung (Flask, SQLAlchemy-Modelle, Routing) • Frontend-Implementierung (Jinja2-Templates, Bootstrap-Layouts) • Befüllen der SQLite-Datenbank mit Testdaten • Meilenstein 2: Abgabe Portfolioteil 2 & Feedback-Schleife

Phase 3: Finalisierung	Woche 7-8	• Erstellung des Testdokuments (Teststrategie, Testprotokolle) • Schreiben der Benutzeranleitung & Erfassen technischer Schulden • Verfassen des 2-seitigen Abstracts („Making-of“ & Reflexion) • Zusammenfügen der Dokumente & ZIP-Export für GitHub • Meilenstein 3: Finale Einreichung der Prüfungsleistung
------------------------	-----------	--

Quelle: eigene Darstellung

2 Anforderungsdokumentation

Das folgende Kapitel erfasst die fachlichen und qualitativen Anforderungen an das System. Neben der Analyse der beteiligten Stakeholder werden die funktionalen Anforderungen als User Stories formuliert sowie die nichtfunktionalen Qualitätskriterien nach ISO/IEC 25010 festgelegt.

2.1 Stakeholder-Analyse

Die Stakeholder-Analyse identifiziert alle Personen, Gruppen und Organisationen, die Einfluss auf das System haben oder von dessen Nutzung direkt oder indirekt betroffen sind. In Tabelle 3 werden die drei wesentlichen Stakeholder-Gruppen hinsichtlich ihrer Systemrolle, ihrer Erwartungshaltung sowie ihrer technischen Vorkenntnisse gegenübergestellt.

Tabelle 3 Stakeholder-Analyse

Stakeholder-Gruppe	Rolle im System	Bedürfnisse & Erwartungen	Technische Vorkenntnisse
Pflegende Angehörige	Hauptnutzende	• Schnelle Erfassung von Medikationsplänen und Vitaldaten • Terminerinnerungen und lückenlose Historie • Automatisierte Erstellung von PDF-Berichten für den Arzt	Grundlegendes IT-Verständnis (Webbrowser / Smartphone)
Zu pflegende Personen	Endnutzende (Patientenansicht)	• Maximale Übersichtlichkeit und Barrierefreiheit • Große Schriften, hohe Kontraste und einfache Interaktion • Klare Fokus-Frage: „Welche Medikamente muss ich jetzt einnehmen?“	Geringe bis sehr geringe IT-Affinität

Ärztliches Fachpersonal	Passiv (empfangend)	• Übersichtliche Zusammenfassung von Vitalwerten (z. B. Blutdruckverläufe) • Nachvollziehbarkeit der Einnahmetreue (Compliance)	Keine direkte Systemnutzung (erhält ausgedrucktes/digitales PDF)
-------------------------	---------------------	--	--

Quelle: eigene Darstellung

Aus dieser Gegenüberstellung wird deutlich, dass das System zwei stark differierende Nutzungskontexte bedienen muss: Während für pflegende Angehörige der Funktionsumfang und die effiziente Datenverwaltung im Vordergrund stehen, erfordert die Interaktion mit den zu pflegenden Personen eine strikte Reduktion der kognitiven Belastung.

2.2 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben das Soll-Verhalten der Anwendung aus Perspektive der jeweiligen Anwenderrollen. Um funktionale Anforderungen aus der Perspektive der jeweiligen Akteure lückenlos und nachvollziehbar zu spezifizieren, wird auf das im Requirements Engineering etablierte Konzept der User Stories zurückgegriffen (Pohl & Rupp, 2021) und diese nach ihrer fachlichen Relevanz für das Minimal Viable Product (MVP) priorisiert.

FA-01: Medikationsverwaltung (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich Medikamente mit Name, Dosierung, Einnahmezeitpunkt und Bestand anlegen, bearbeiten und löschen, um den Medikationsplan stets aktuell zu halten.“

Begründung & Interaktion: Im Pflegealltag führen unvollständige oder veraltete Medikationspläne häufig zu Medikationsfehlern. Diese Anforderung stellt sicher, dass alle relevanten Parameter eines Arzneimittels zentral hinterlegt werden. Der Nutzer interagiert über ein strukturiertes Formular mit dem System, das Pflichtfelder für Name, Dosierung (z. B. „1 Tablette“) und Einnahmezeitpunkte (z. B. „Morgens“) erzwingt.

FA-02: Vitaldaten-Tracking (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich tägliche Vitalwerte (Blutdruck, Puls, Gewicht) mit Zeitstempel erfassen und in Tabellen einsehen, um gesundheitliche Trends frühzeitig zu erkennen.“

Begründung & Interaktion: Die Erfassung von Vitalwerten liefert wertvolle Indikatoren für den physischen Zustand der zu pflegenden Person. Das System stellt spezialisierte Eingabemasken bereit, die Messwerte mit einem automatischen Zeitstempel versehen. Die erfassten Daten werden in einer chronologischen Übersicht aufbereitet, um kritische Abweichungen schnell zu identifizieren.

FA-03: Arzt-Bericht / PDF-Export (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich gefilterte Vital- und Medikationsdaten für einen wählbaren Zeitraum als PDF exportieren, um diese beim Hausarztbesuch vorzulegen.“

Begründung & Interaktion: Konsultationen bei behandelnden Ärztinnen und Ärzten leiden oft unter Zeitmangel und unvollständigen Patientenauskünften. Durch diese Funktion generiert das System auf Knopfdruck ein komprimiertes, übersichtliches PDF-Dokument. Die anwendende Person wählt lediglich den gewünschten Zeitbereich aus; das Backend aggregiert daraufhin die Daten automatisch.

FA-04: Termin- und Aufgabenverwaltung (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich anstehende Pflegetermine sowie Aufgaben eintragen, um den Pflegealltag strukturiert zu organisieren.“

Begründung & Interaktion: Die Koordinierung von Arztbesuchen, Physiotherapie und pflegerischen Aufgaben erfordert eine verlässliche Alltagsstruktur. Die Anwendung bietet ein einfaches Aufgaben-Board, in dem Termine mit Datum, Uhrzeit und Kategorie erfasst und nach Erledigung abgehakt werden können.

FA-05: Barrierefreie Tagesansicht / Patientenmodus (Priorität: Hoch)

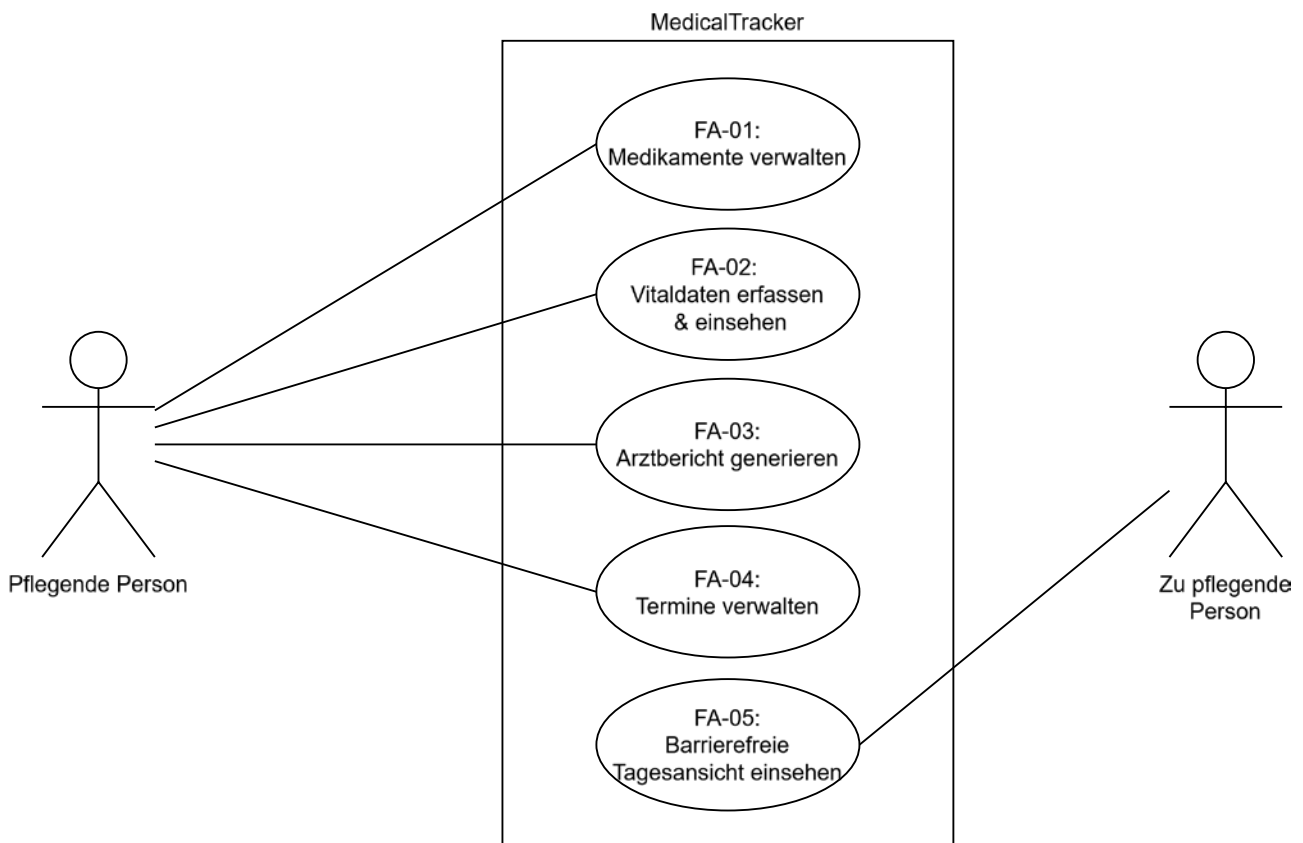
User Story: „Als zu pflegende Person möchte ich eine vereinfachte Tagesansicht aufrufen können, die mir in großer Schrift nur die heute anstehenden Medikamente anzeigt, um Orientierung im Alltag zu erhalten.“

Begründung & Interaktion: Zur Förderung der Eigenständigkeit von Patientinnen und Patienten verzichtet diese Ansicht auf komplexe Navigationsstrukturen. Über großflächige Schaltflächen und eine reduzierte Informationsdichte sieht die anwendende Person ausschließlich die für den aktuellen Tag relevanten Einnahmen.

2.3 UML-Anwendungsfalldiagramm

Zur visuellen Abgrenzung der Systemgrenzen sowie zur Verdeutlichung der Interaktionen zwischen den identifizierten Akteuren und den funktionalen Anforderungen dient das in Abbildung 1 dargestellte UML-Anwendungsfalldiagramm.

Abbildung 1 Anwendungsfalldiagramm



Quelle: eigene Darstellung

Aus der Modellierung geht klar die funktionale Rollentrennung hervor: Während die pflegende Person als Hauptakteur Vollzugriff auf alle administrativen Verwaltungs- und Analysefunktionen (FA-01 bis FA-04) besitzt, beschränkt sich die Interaktion der zu pflegenden Person gezielt auf den barrierefreien Patientenmodus (FA-05). Dies unterstützt das Ziel der kognitiven Entlastung der zu pflegenden Person.

2.4 Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen definieren die Qualitätskriterien, Rahmenbedingungen und Performanz-Grenzen, die das System einhalten muss. Zur wissenschaftlichen Fundierung orientiert sich die Klassifikation an der Norm ISO/IEC 25010 (ISO/IEC, 2023). Tabelle 4 führt die nichtfunktionalen Anforderungen auf.

Tabelle 4 Nichtfunktionale Anforderungen

NFA-ID	Kategorie	Anforderung & Spezifikation
NFA-01	Nutzbarkeit (Interaction Capability)	Die Gestaltung des Patientenmodus orientiert sich an den Richtlinien der Barrierefreiheit der WCAG 2.2 (W3C, 2024). Es wird ein hohes Kontrastverhältnis eingehalten. Für den Patientenmodus werden großflächige Interaktionselemente sowie eine projektspezifische Basis-Schriftgröße von mindestens 18 pt vorgegeben.
NFA-02	Leistungsfähigkeit (Performance Efficiency)	Die Antwortzeit des lokalen Flask-Servers darf beim Laden von Ansichten unter Windows 10/11 maximal 1 Sekunde betragen. Lese- und Schreibzugriffe auf die SQLite-Datenbank müssen unter 200 ms liegen.
NFA-03	Sicherheit (Security)	Alle personenbezogenen Gesundheitsdaten verbleiben ausschließlich in der lokalen SQLite-Datenbank (Privacy by Design). Formulardaten werden gegen SQL-Injection (via SQLAlchemy ORM) und XSS (via Jinja2 Auto-Escaping) geschützt.
NFA-04	Zuverlässigkeit (Reliability)	Das System darf bei fehlerhaften Benutzereingaben (z. B. Buchstaben im Blutdruckfeld) nicht abstürzen. Fehleingaben werden abgefangen und durch visuelle Hinweise im Formular quittiert.
NFA-05	Flexibilität (Flexibility)	Die Anwendung muss ohne externe Serverinfrastruktur nativ im Standard-Webbrowser unter Windows 10/11 lauffähig sein. Ein lokaler Python-Interpreter (v3.13+) ist als Ausführungsumgebung ausreichend.

Quelle: eigene Darstellung

3 Spezifikationsdokumentation

Dieses Kapitel beschreibt das objektorientierte Datenmodell der zentralen Geschäftsobjekte sowie deren fachliche Beziehungen und Kardinalitäten. Des Weiteren werden die dynamischen Geschäftsprozesse – mit besonderem Fokus auf den Kernprozess der Medikamenteneinnahme und Bestandsführung – detailliert dargelegt. Ergänzend werden die strikt einzuhaltenden Geschäftsregeln formuliert sowie die technischen System- und Benutzerschnittstellen spezifiziert.

3.1 Objektorientiertes Datenmodell & Geschäftsobjekte

Das Datenmodell definiert die zentralen Geschäftsobjekte (Business Objects) der Anwendung, deren Fachattribute sowie die bestehenden Beziehungsstrukturen untereinander.

1. Geschäftsobjekt: Patient

Repräsentiert die zu pflegende Person als zentralen Bezugspunkt aller Datensätze.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- vorname (String): Vorname der zu pflegenden Person.
- nachname (String): Nachname der zu pflegenden Person.
- geburtsdatum (Date): Geburtsdatum.
- notfallkontakt (String): Telefonnummer oder Name einer Kontaktperson für Notfälle.

2. Geschäftsobjekt: Medikament

Repräsentiert ein verordnetes Arzneimittel inklusive Einnahmевorschrift und Bestandsführung.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- name (String): Handelsname des Medikaments (z. B. "Ramipril 5mg").
- dosierung (String): Verordnete Einnahmemenge (z. B. "1 Tablette").
- tageszeit (String): Vorgesehener Einnahmezeitpunkt (z. B. "Morgens").
- bestand (Integer): Aktuell vorhandener Vorrat in Einheiten.
- mindestbestand (Integer): Schwellenwert für automatische Nachbestell-Warnungen.

3. Geschäftsobjekt: Vitalwert

Erfasst eine einzelne medizinische Messung zu einem konkreten Zeitpunkt.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- zeitpunkt (DateTime): Datum und Uhrzeit der Messung.
- kategorie (String): Art der Messung (z. B. "Blutdruck Systolisch", "Puls", "Gewicht").
- wert (Float): Gemessener numerischer Wert.
- einheit (String): Maßeinheit des Werts (z. B. "mmHg", "bpm", "kg").

4. Geschäftsobjekt: Termin

Speichert geplante Pflegetermine und Aufgaben im Alltagsplaner.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- titel (String): Kurzbeschreibung des Termins (z. B. "Arzttermin Dr. Weber").
- zeitpunkt (DateTime): Ansetzungszeitpunkt.
- kategorie (String): Art des Termins (z. B. "Arzt", "Physiotherapie").
- erledigt (Boolean): Bearbeitungsstatus (Ja/Nein).

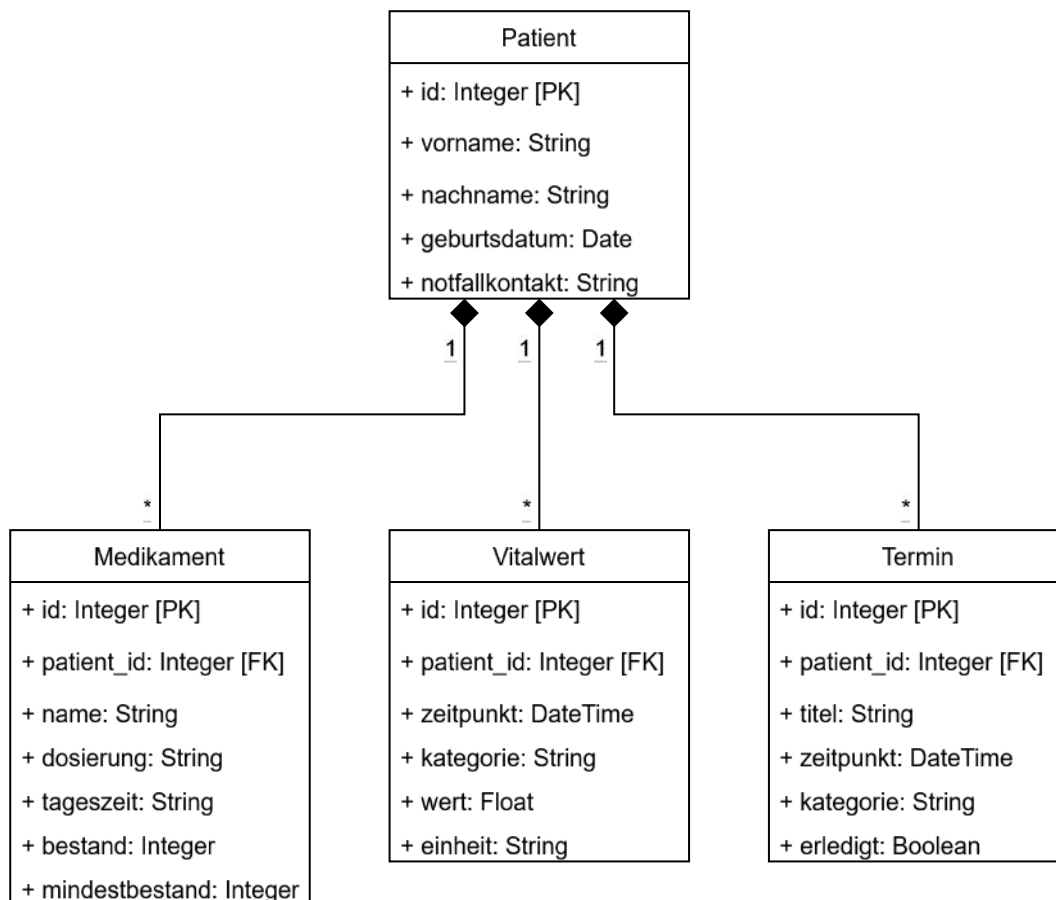
Multiplizitäten & Fachliche Beziehungen

Zwischen den Geschäftsobjekten bestehen klare 1:n-Kardinalitäten:

- Ein Patient besitzt beliebig viele zugeordnete Medikamente.
- Ein Patient besitzt beliebig viele historische Vitalwerte.
- Ein Patient besitzt beliebig viele geplante Termine.

Die statische Struktur des Datenmodells sowie die fachlichen Beziehungsstrukturen und Kardinalitäten zwischen den zentralen Geschäftsobjekten werden im nachfolgenden UML-Klassendiagramm (Abbildung 2) zusammenfassend abgebildet.

Abbildung 2 Klassendiagramm



Quelle: eigene Darstellung

Das Klassendiagramm verdeutlicht die zentrale Stellung des Geschäftsobjekts Patient. Die abhängigen Entitäten Medikament, Vitalwert und Termin stehen jeweils über fremdschlüsselbasierte 1:n-Kardinalitäten in direkter Relation zum Patienten. Diese Struktur garantiert, dass alle Gesundheits- und Pflegedaten im System eindeutig einer Person zugeordnet sind.

3.2 Geschäftsprozesse

Die Geschäftsprozesse beschreiben das dynamische Zusammenspiel zwischen Benutzereingaben, Systemlogik und Datenhaltung.

GP-01: Erfassung eines neuen Medikaments

Der Prozess beginnt mit dem Aufruf der Medikationsübersicht durch den pflegenden Angehörigen. Nach Klick auf die Schaltfläche „Neues Medikament“ öffnet sich ein Formular. Die anwendende Person gibt die Stammdaten (Name, Dosierung, Einnahmezeitpunkt) sowie den aktuellen Anfangsbestand und den gewünschten Mindestbestand ein. Bei der Übermittlung prüft das System die Eingaben auf Vollständigkeit und numerische Plausibilität. Nach erfolgreicher Validierung wird der Datensatz persistent in der SQLite-Datenbank gespeichert und die anwendende Person erhält eine Bestätigungsmeldung.

GP-02: Vitaldatenerfassung und Schwellenwertprüfung

Zur Dokumentation von Vitalwerten wählt die anwendende Person die entsprechende Kategorie (z. B. Blutdruck) aus. Nach der Werteingabe führt das System eine client- und serverseitige Validierung durch, um fehlerhafte Datentypen (z. B. Texteingaben) abzufangen. Ergänzend wird geprüft, ob die eingegebenen Werte innerhalb medizinisch plausibler Grenzen liegen. Liegt beispielsweise der systolische Blutdruckwert unter dem diastolischen Wert, bricht das System den Speichervorgang ab und gibt eine detaillierte Fehlermeldung aus. Validierte Werte werden zusammen mit dem aktuellen Serverzeitstempel in der Historie abgelegt.

GP-03: Kernprozess – Medikamenteneinnahme und Bestandsaktualisierung

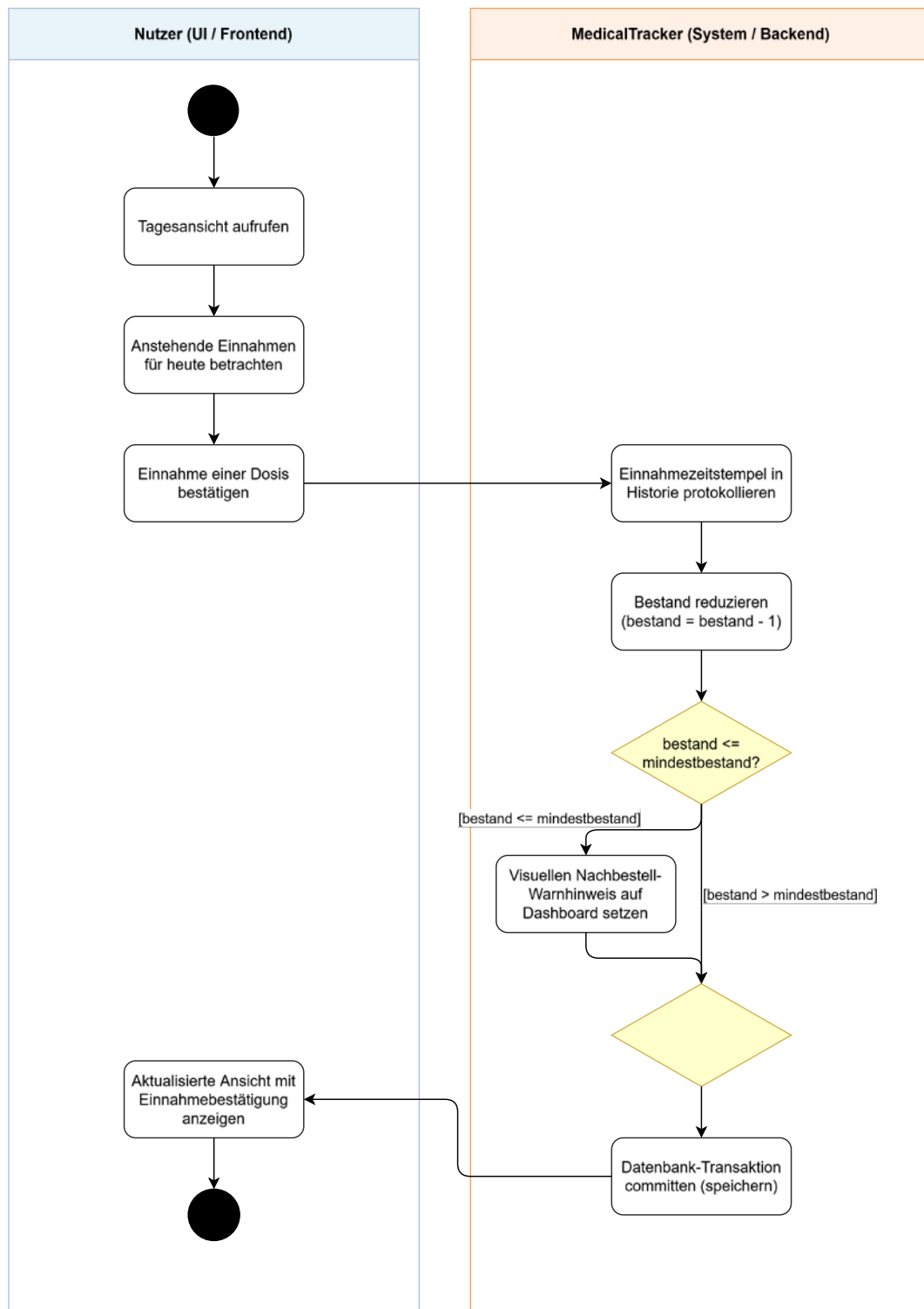
Der zentrale Geschäftsprozess verknüpft die alltägliche Pflegeinteraktion direkt mit der automatisierten Bestandsführung des Systems. Das Ablaufmuster gliedert sich in folgende Schritte:

1. **Prozessstart:** Die pflegende oder zu pflegende Person öffnet die barrierefreie Tagesansicht. Das System liest die für den aktuellen Tag terminierten Einnahmen aus der Datenbank aus und stellt diese dar.
2. **Einnahmebestätigung:** Die anwendende Person markiert eine anstehende Dosis durch Anklicken der entsprechenden Bestätigungsschaltfläche als „eingenommen“.
3. **Zustandsänderung im Backend:** Das Backend nimmt den HTTP-Request entgegen und initiiert eine Datenbanktransaktion. Der Status der spezifischen Einnahme für den Tag wird auf erledigt gesetzt. Zudem wird der Gesamtwert des Attributs bestand um die verabreichte Menge dekrementiert.
4. **Verzweigung & Bestandsprüfung:** Das System führt eine logische Verzweigung durch und prüft, ob der tatsächliche Bestand größer oder gleich dem Mindestbestand ist.
 - a. **Schwellenwert unterschritten:** Der verbleibende Bestand ist kleiner oder gleich dem definierten Mindestbestand. Das System setzt ein internes Flag und generiert einen visuellen Warnhinweis auf dem Dashboard.

- b. Bestand ausreichend: Der Bestand liegt über dem Schwellenwert. Es wird kein Warnhinweis erzeugt.
5. Transaktionsabschluss: Die Änderung wird in der Datenbank gespeichert (Commit) und die Ansicht wird mit den neuen Werten aktualisiert.

Um den dynamischen Ablauf des Kernprozesses GP-03 inklusive der Kontrollflüsse, Systemgrenzen und Entscheidungslogiken nachvollziehbar darzustellen, wird dieser im nachfolgenden UML-Aktivitätsdiagramm (Abbildung 3) in die beiden Swimlanes Frontend (Nutzer) und Backend (System) unterteilt.

Abbildung 3 Aktivitätsdiagramm



Quelle: eigene Darstellung

Wie die Prozessmodellierung zeigt, koppelt das System die Interaktion im Frontend direkt mit einer automatisierten Bestandskontrolle im Backend. Durch die Auswertung der Bedingung wird sichergestellt, dass bei kritischen Beständen unverzüglich eine optische Nachbestell-Warnung auf dem Dashboard generiert wird, bevor die Datenbanktransaktion erfolgreich abgeschlossen wird.

3.3 Geschäftsregeln

Geschäftsregeln definieren fachliche Bedingungen und Invarianten, die vom System jederzeit durchgesetzt werden müssen. Um Fehleingaben im Pflegealltag zu vermeiden und eine verlässliche Datenbasis für behandelnde Fachkräfte zu garantieren, werden die Regeln über Prüfroutinen im Frontend, Validierungslogiken im Backend und Constraints in der SQLite-Datenbank abgesichert. Um eine maximale Datenintegrität zu garantieren, werden die Geschäftsregeln nach dem Prinzip der mehrstufigen Validierung verankert. Dies umfasst Prüfroutinen auf der Benutzeroberfläche, Validierungslogiken in den Flask-Routen sowie harte Integritätsbedingungen innerhalb des SQLite-Datenbankschemas. Nachfolgend werden die zentralen Geschäftsregeln des Systems detailliert spezifiziert:

GR-01 (Bestandspositivität):

Formale Bedingung:

$$\text{bestand} \geq 0 \wedge \text{mindestbestand} \geq 0$$

Begründung:

Ein negativer Arzneimittelbestand ist physikalisch unmöglich. Negative Werte würden die automatisierte Nachbestell-Logik verfälschen und das Generieren von Warnmeldungen auf dem Dashboard blockieren oder fehlerhafte Signale auslösen.

Technische Durchsetzung:

- Clientseitig: HTML5-Eingabefelder mit dem Attribute `min="0"`.
- Serverseitig: Validierung vor Ausführung des Datenbank-Commits.
- Datenbankebene: Verankerung einer `CheckConstraint('bestand >= 0')` im SQLAlchemy-Modell der Entität `Medikament`.

GR-02 (Blutdruck-Plausibilität):

Formale Bedingung:

$$\text{systolisch} > \text{diastolisch} \wedge (30 \text{ mmHg} \leq \text{diastolisch} < \text{systolisch} \leq 300 \text{ mmHg})$$

Begründung:

Aus physiologischer Sicht beschreibt der systolische Wert den maximalen Druck während der Kontraktionsphase des Herzens, während der diastolische Wert den Dauerdruck in der Entspannungsphase wiedergibt (McEvoy et al., 2024). Ein Vertauschen beider Werte würde die automatische Verlaufsanalyse für das behandelnde Fachpersonal verfälschen.

Die Schranken von 30 mmHg bis 300 mmHg stellen ein softwareseitiges Validierungsfenster dar. Es deckt das Spektrum extremer klinischer Zustände, von schweren Schockzuständen bis hin zu hypertensiven Notfällen, ab und orientiert sich an den technischen Messbereichen nach DIN EN ISO 81060-2 (Deutsches Institut für Normung, 2024) für nicht-invasive Blutdruckmessgeräte. Werte außerhalb dieses Fensters weisen mit an Sicherheit grenzender Wahrscheinlichkeit auf Fehleingaben (z. B. Tippfehler) hin.

Technische Durchsetzung:

- Serverseitig: Eine benutzerdefinierte Validierungsfunktion im Flask-Backend prüft bei der Übermittlung von Blutdruck-Datensätzen die Relation beider Werte.
- Fehlerbehandlung: Bei Regelverletzung wird die Transaktion abgebrochen, ein Rollback ausgeführt und das Formular mit einem spezifischen Hinweistext („Der systolische Wert muss höher als der diastolische Wert sein“) neu gerendert.

GR-03 (Historiensicherheit):

Formale Bedingung:

$\forall x \in \{\text{Vitalwerte}, \text{Einnahmeprotokolle}\}: \text{UPDATE}(x) = \perp \wedge \text{DELETE}(x) = \perp$

Begründung:

Einmal erfasste Vitaldaten und Einnahmebestätigungen bilden die chronologische Dokumentationsgrundlage für die behandelnde Ärzteschaft. Das nachträgliche Manipulieren oder Stornieren historischer Einträge würde die Nachvollziehbarkeit der Einnahmetreue (Compliance) gefährden und den Wert des generierten Arztberichts entkräften.

Technische Durchsetzung:

- Architektonisch: Auf API- und Routing-Ebene werden für Protokolleinträge und historische Vitalwerte keine UPDATE- oder DELETE-Endpunkte für Standard-Anwender bereitgestellt.
- Datenbankebene: Neue Messungen werden ausschließlich über INSERT-Operationen als neue Datensätze angefügt.

GR-04 (Geburtsdatum):

Formale Bedingung:

geburtsdatum < *datum*_{aktuell}

Begründung:

Das Geburtsdatum der zu pflegenden Person dient als Stammdatenbasis für die Altersberechnung und Identifikation. Ein in der Zukunft liegendes Datum ist logisch ausgeschlossen.

Technische Durchsetzung:

- Clientseitig: Einschränkung des HTML5-Datepickers durch das dynamisch gesetzte Attribut `max="YYYY-MM-DD"`.
- Serverseitig: Vergleichende Prüfung gegen `datetime.date.today()` in der Python-Geschäftslogik vor dem Speichern der Patientendaten.

3.4 Schnittstellen & Validierung

Schnittstellen bilden die definierten Übergabepunkte eines Softwaresystems zu seiner Umwelt. Sie gliedern sich in Systemschnittstellen zur technischen Kommunikation zwischen Softwarekomponenten und Benutzerschnittstellen zur Human-Computer-Interaction. Die Schnittstellen stellen sicher, dass Daten sowohl technisch sicher verarbeitet werden, als auch für zwei stark unterschiedliche Anwendergruppen ergonomisch zugänglich sind.

3.4.1 Systemschnittstellen

Die internen und externen Systemschnittstellen regeln den Datenaustausch zwischen den Komponenten der Anwendungsarchitektur. Sie stellen sicher, dass Informationen verlustfrei, typsicher und performant übertragen werden.

Internes HTTP-REST-Kommunikationsmodell (Frontend ↔ Backend)

Zweck:

Übertragung von Benutzereingaben, Formularinhalten und Navigationsbefehlen zwischen dem Browser des Anwenders und dem Flask-Backend.

Protokoll & Methoden:

Standardisiertes HTTP/1.1 via GET- und POST-Requests.

Datenformat:

Formulardaten werden als `application/x-www-form-urlencoded` an den Server übermittelt. Bei dynamischen UI-Aktualisierungen (z. B. Statusänderungen in der Tagesansicht) werden serverseitig gerenderte HTML-Snippets oder strukturelle JSON-Antworten (`application/json`) ausgetauscht.

PDF-Reporting-Schnittstelle (Backend ↔ Export-Engine)

Zweck:

Dynamische Erzeugung von aufbereiteten Arztberichten auf Basis historischer Datenbankeinträge.

Protokoll & Bibliotheken:

Anbindung der Python-Bibliothek ReportLab über eine interne Service-Schnittstelle.

Datenformat & Ablauf:

Die Schnittstelle nimmt aggregierte Datensätze (Vitalwerte, Medikationshistorie) entgegen, verarbeitet diese über eine Template-Engine und generiert als binären Datenstrom ein valides PDF-Dokument (`application/pdf`), das über den HTTP-Header `Content-Disposition: attachment` als Download bereitgestellt wird.

Objekt-Relationale Datenbankschnittstelle (Backend ↔ SQLite)

Zweck:

Entkopplung der objektorientierten Python-Anwendungslogik vom relationalen Datenbankmodell.

Protokoll & Technologie:

Kommunikation über den Objekt-Relationalen Mapper Flask-SQLAlchemy und den nativen `sqlite3`-Treiber.

Funktionsweise:

Anfragen werden im Python-Code als objektorientierte Abfragen formuliert und vom ORM automatisch in optimierte SQL-Statements (`SELECT`, `INSERT`, `UPDATE`) übersetzt. Dies schützt das System architektonisch vor SQL-Injection-Angriffen.

3.4.2 Benutzerschnittstellen & UI-Konzept

Die Gestaltung der Benutzeroberfläche folgt einem dualen Schnittstellenkonzept, das den stark voneinander abweichenden Anforderungen der beiden primären Stakeholder-Gruppen Rechnung trägt.

Dashboard für pflegende Angehörige (Verwaltungsmodus)

Zielgruppe:

Pflegende Angehörige mit durchschnittlichen IT-Vorkenntnissen.

Struktur & Layout:

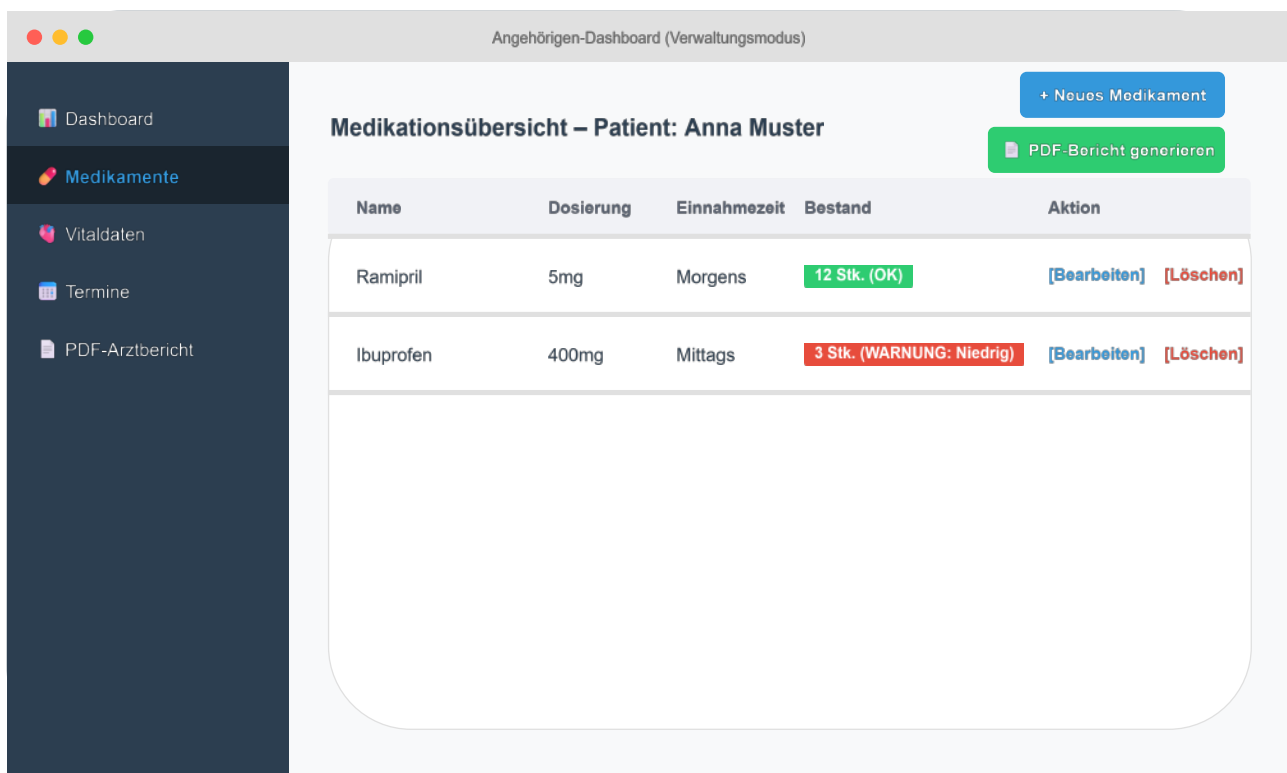
Die Oberfläche basiert auf einem responsiven Dashboard-Layout mit fixer Sidebar-Navigation. Das Design bietet eine hohe Informationsdichte und ermöglicht die effiziente Verwaltung aller Daten.

Interaktionselemente:

Tabellarische Ansichten zur Anzeige von Medikations- und Vitaldaten-Historien, Filter- und Suchfunktionen sowie modale Dialogfenster zur schnellen Erfassung neuer Datensätze, ohne die aktuelle Seite verlassen zu müssen.

Das grundlegende Verteilungsmuster und die funktionale Struktur des Verwaltungsdashboards werden in der nachfolgenden Skizze (Abbildung 4) dargestellt.

Abbildung 4 Verwaltungsdashboard



Quelle: eigene Darstellung

Das Mock-up verdeutlicht die klare Trennung von Hauptnavigation (links) und Arbeitsbereich (rechts). Durch den gezielten Einsatz von Tabellen-Aktionen und prominent platzierten Aktions-Buttons wird eine intuitive Bedienbarkeit bei administrativen Aufgaben gewährleistet.

Patientenmodus (Barrierefreie Tagesansicht)

Zielgruppe:

Zu pflegende Personen mit potenziell motorischen oder visuellen Einschränkungen und geringer IT-Affinität.

Struktur & Layout:

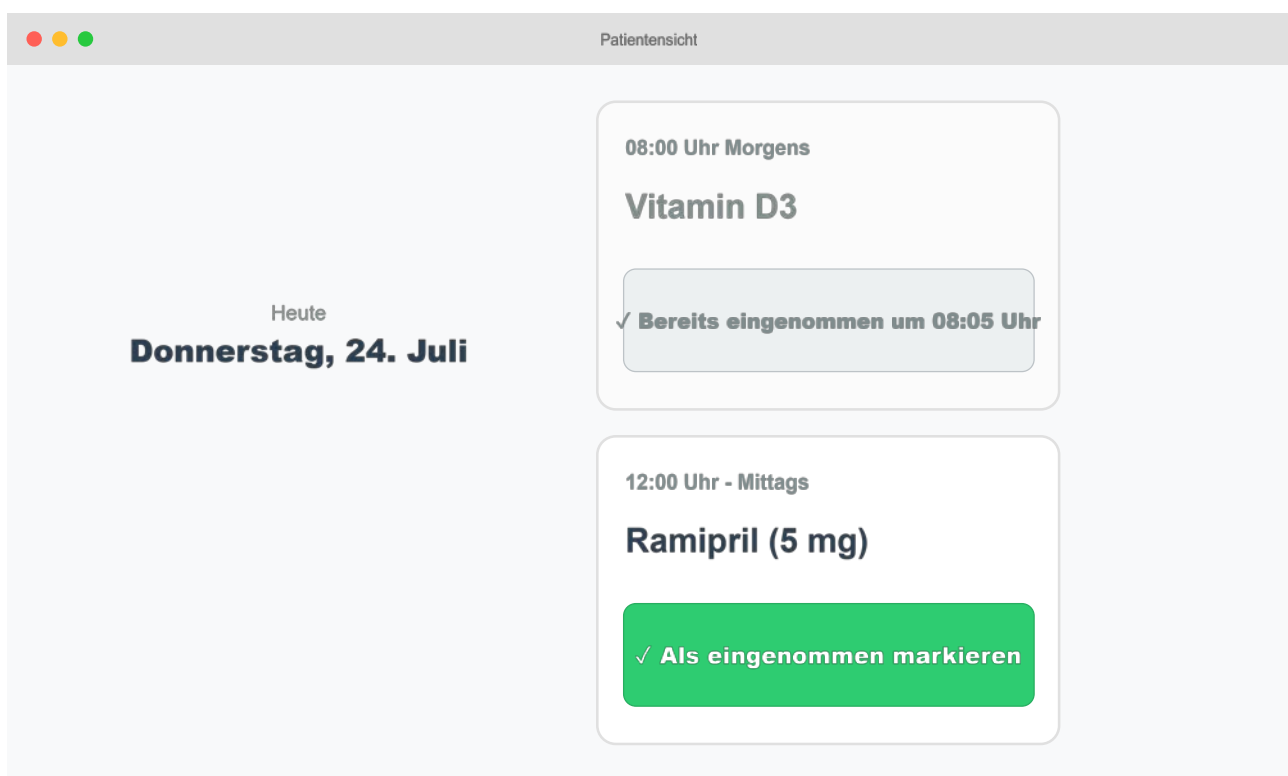
Kognitiv stark reduziertes Single-Page-Design. Es verzichtet auf störende Navigationsleisten, verschachtelte Menüs und Werbeelemente.

Interaktionselemente & Ergonomie: Typografie:

Große, gut lesbare Sans-Serif-Schriftarten mit einer Basis-Schriftgröße von mindestens 18pt. Kontraststarke Farbgestaltung. Großflächige Touch-Buttons zur einfachen Bestätigung der Medikamenteneinnahme.

Die reduzierte und barrierefreie Benutzeroberfläche des Patientenmodus ist in Abbildung 5 illustriert.

Abbildung 5 Patientensicht



Quelle: eigene Darstellung

Wie in Abbildung 5 erkennbar ist, konzentriert sich die Oberfläche ausschließlich auf die zentralen Fragen der zu pflegenden Person („Was muss ich jetzt einnehmen?“). Die Interaktion beschränkt sich auf das einfache Bestätigen über großflächige Schalter, was die Handlungsautonomie des Patienten fördert.

3.4.3 Mehrstufiges Validierungskonzept

Um Fehleingaben, unvollständige Formulare oder bösartige Eingaben wirksam abzufangen und die Systemstabilität zu garantieren, setzt die Anwendung auf ein mehrstufiges Validierungskonzept:

Clientseitige Validierung (HTML5 & Browser)

Bereits im Frontend werden Formulareingaben durch native HTML5-Attribute vorab geprüft. Attribute wie `required` (Pflichtfeld), `type="number"` (Erkennen von Falschformaten), `min="0"` (Verhinderung negativer Bestände) sowie `step="0.1"` bieten dem Nutzer eine unmittelbare Rückmeldung noch vor dem Absenden der Daten.

Serverseitige Validierung (Flask & Python)

Da clientseitige Prüfungen im Webbrowser umgangen werden können, führt das Flask-Backend bei jedem eingehenden Request eine erneute Prüfung aller Parameter durch. Datentypen werden explizit konvertiert (z. B. String zu Float) und fachliche Grenzwerte (gemäß den Geschäftsregeln aus Kapitel 3.3) evaluiert.

Fehlerbehandlung und Transaktionssicherheit

Schlägt die Validierung im Backend fehl, wird die angeforderte Datenbankoperation sofort abgebrochen und ein Rollback der Transaktion eingeleitet. Dem Anwender wird das Ursprungsformular mit einer spezifischen, gut verständlichen Fehlermeldung neu gerendert, sodass keine unvollständigen oder fehlerhaften Daten im System verbleiben.

Literaturverzeichnis

Deutsches Institut für Normung. (2024). *Nichtinvasive Blutdruckmessgeräte — Teil 2: Klinische Prüfung der intermittierenden automatisierten Bauart* (DIN EN ISO 81060-2:2024-08)

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software quality models* (ISO/IEC Standard No. 25010:2023).

McEvoy, J. W., McCarthy, C. P., Bruno, R. M., Brouwers, S., Canavan, M. D., Ceconi, C., Christodorescu, R. M., Daskalopoulou, S. S., Ferro, C. J., Gerds, E., Hanssen, H., Harris, J., Lauder, L., McManus, R. J., Molloy, G. J., Rahimi, K., Regitz-Zagrosek, V., Rossi, G. P., Sandset, E. C., . . . Khamidullaeva, G. A. (2024). 2024 ESC Guidelines for the management of elevated blood pressure and hypertension. *European Heart Journal*, 45(38), 3912–4018. <https://doi.org/10.1093/eurheartj/ehae178>

Pohl, K. & Rupp, C. (2021). *Basiswissen Requirements Engineering: Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level*.

W3C. (2023). *Web content accessibility guidelines (WCAG) 2.2* (W3C Recommendation). W3C.